

Probability_Stats

April 13, 2026

1 Probability and Statistics

1.1 Contents

- §9.1 Probability
 - Events
 - Conditional Probability
 - Random Variables
 - Indicator Random Variable
 - Expected Value
 - Conditional Expectation
 - Variance
 - Standard Deviation
 - Covariance
 - Covariance Matrix
 - Correlation Coefficient
 - Entropy
- Distributions
 - Probability Mass Function (PMF)
 - Cumulative Distribution Function (CDF)
 - Probability Density Function (PDF)
 - Identically Distributed and IID
 - Binomial Distribution
 - Poisson Distribution
 - Exponential Distribution
 - Normal / Gaussian Distribution
 - Chi-Squared Distribution
 - Student's t-Distribution
 - F-Distribution
- Convergence of Random Variables
 - Convergence in Distribution
 - Convergence in Probability
 - Almost Sure Convergence
 - L^2 Convergence
- §9.2 Statistics
 - Average
 - Median
 - Sample Standard Deviation

- z-Score
- Order Statistics
- Hypothesis Testing
- Error Terms
- Estimator

1.2 Notes

- The notebooks are largely self-contained. If a symbol appears, an explanation will be present at some point in the same notebook — most often with a link to the cell where it is explained. Only if a symbol cannot be covered in this notebook will a reference to another notebook be provided.
- **GitHub does a poor job of rendering these notebooks.** The online render is missing links, symbols are absent, and notations are badly formatted. Clone a local copy or download the notebook and open it locally, or refer to the PDFs in the PDFs/ folder.
- All code uses only Python's standard library.
- The `random` module is used with fixed seeds so that simulation output is reproducible.

See the [Collections](#) notebook for familiarity with set notation and the [Numbers](#) notebook for number types.

1.3 Importing Libraries

```
[1]: import math
import random
```

§9.1 Probability

In probability theory, one begins with a set of fundamental outcomes, often denoted Ω or S . An *event* is a subset of Ω . The probability of an event A is typically denoted $P(A)$. The notations $\Pr(A)$ and $\mathbb{P}(A)$ are also used.

Events

Given a sample space Ω and two events $A, B \subseteq \Omega$, the following notations are standard:

- $P(A \cap B)$ is the probability of A *and* B . It is also sometimes denoted $P(AB)$ or $P(A \wedge B)$.
- $P(A \cup B)$ is the probability of A *or* B . It is also sometimes denoted $P(A \vee B)$.

For finite, equally likely sample spaces, the probability of an event is:

$$P(A) = \frac{|A|}{|\Omega|}$$

```
[2]: omega = {1, 2, 3, 4, 5, 6}
A = {2, 4, 6}
B = {1, 2, 3}
print('P(A) =', len(A) / len(omega))
```

```
print('P(B) =', len(B) / len(omega))
```

P(A) = 0.5

P(B) = 0.5

```
[3]: omega = {1, 2, 3, 4, 5, 6}
A = {2, 4, 6}
B = {1, 2, 3}
print('P(A and B) =', len(A & B) / len(omega))
print('P(A or B) =', len(A | B) / len(omega))
```

P(A and B) = 0.16666666666666666

P(A or B) = 0.8333333333333334

Conditional Probability

The conditional probability of event A given event B is denoted $P(A | B)$ and is defined as:

$$P(A | B) = \frac{P(A \cap B)}{P(B)}$$

This measures the probability of A occurring, given that B has occurred.

```
[4]: omega = {1, 2, 3, 4, 5, 6}
A = {2, 4, 6}
B = {1, 2, 3}
P_A_given_B = len(A & B) / len(B)
P_A_given_B
```

```
[4]: 0.3333333333333333
```

Random Variables

Random variables (often abbreviated rv) are generally denoted with capital letters. The notation $X(\omega)$ is the value of X at the point ω of the underlying probability space on which X is defined. Notation such as $P(X \geq 0)$ is a shorthand for $P(\{\omega : X(\omega) \geq 0\})$.

```
[5]: omega = {1, 2, 3, 4, 5, 6}
X = {w: w for w in omega}
# P(X >= 4) = P({w : X(w) >= 4})
favorable = {w for w in omega if X[w] >= 4}
len(favorable) / len(omega)
```

```
[5]: 0.5
```

Indicator Random Variable

Given an event A , the notation $\mathbf{1}_A$ is an *indicator* random variable defined by:

$$\mathbf{1}_A(\omega) = \begin{cases} 1 & \text{if } \omega \in A \\ 0 & \text{otherwise} \end{cases}$$

Some authors write I_A for indicator random variables.

```
[6]: A = {2, 4, 6}
I_A = [1 if w in A else 0 for w in range(1, 7)]
print('omega:', list(range(1, 7)))
print('I_A: ', I_A)
```

```
omega: [1, 2, 3, 4, 5, 6]
```

```
I_A:    [0, 1, 0, 1, 0, 1]
```

Expected Value

For a random variable X , its *expected value* is denoted $E(X)$. Sometimes a different style is used, e.g., $\mathcal{E}(X)$ or $\mathbb{E}(X)$. The letter μ (standing for *mean*) is often used for the expected value. If more than one random variable is under consideration, one may subscript μ with the name of the random variable: μ_X is the expected value of X .

In some disciplines, the notation $\langle X \rangle$ is used to denote the expected value of X .

For a discrete random variable taking values $x_1, x_2, \dots, x_n$ with equal probability:

$$E(X) = \frac{1}{n} \sum_{i=1}^n x_i$$

```
[7]: X = [1, 2, 3, 4, 5, 6]
p = 1 / len(X)
E_X = sum(x * p for x in X)
E_X
```

```
[7]: 3.5
```

Conditional Expectation

The conditional expected value of X given Y is denoted $E(X \mid Y)$. This is also written $\mu_{X|Y}$. It represents the expected value of X when the value of Y is known.

```
[8]: X = [1, 2, 3, 4, 5, 6]
# E(X | X > 3): restrict to outcomes where X > 3
B = [x for x in X if x > 3]
E_X_given_B = sum(B) / len(B)
E_X_given_B
```

[8]: 5.0

Variance

For a random variable X , the *variance* of X is denoted $\text{Var}(X)$ and defined by:

$$\text{Var}(X) = E[(X - \mu)^2] = E(X^2) - E(X)^2$$

The variance is often denoted σ^2 . It may be subscripted with the name of the random variable: σ_X^2 .

```
[9]: X = [1, 2, 3, 4, 5, 6]
mu = sum(X) / len(X)
Var_X = sum((x - mu)**2 for x in X) / len(X)
print('E(X) =', mu, ', Var(X) =', round(Var_X, 4))
```

E(X) = 3.5 , Var(X) = 2.9167

Standard Deviation

The square root of the variance is known as the *standard deviation* and is denoted σ :

$$\sigma = \sqrt{\text{Var}(X)}$$

```
[10]: X = [1, 2, 3, 4, 5, 6]
mu = sum(X) / len(X)
sigma = math.sqrt(sum((x - mu)**2 for x in X) / len(X))
sigma
```

[10]: 1.707825127659933

Covariance

Given two random variables X and Y , their *covariance* is:

$$\text{Cov}(X, Y) = E(XY) - E(X)E(Y)$$

A positive covariance indicates that X and Y tend to increase together; a negative covariance indicates an inverse relationship.

```
[11]: X = [1, 2, 3, 4, 5]
Y = [2, 4, 5, 4, 5]
n = len(X)
E_XY = sum(X[i] * Y[i] for i in range(n)) / n
Cov_XY = E_XY - (sum(X) / n) * (sum(Y) / n)
```

Cov_XY

```
[11]: 1.1999999999999993
```

Covariance Matrix

If $\mathbf{X}$ is a random vector,

$$\mathbf{X} = \begin{bmatrix} X_1 \\ X_2 \\ \vdots \\ X_n \end{bmatrix}$$

then the *covariance matrix* of $\mathbf{X}$ is often denoted Σ . This is an $n \times n$ matrix whose i, j -entry is $\text{Cov}(X_i, X_j)$.

```
[12]: X1 = [1, 2, 3, 4, 5]
      X2 = [2, 4, 5, 4, 5]
      n = len(X1)
      cov = lambda A, B: sum(A[i]*B[i] for i in range(n))/n - (sum(A)/n)*(sum(B)/n)
      Sigma = [[round(cov(a, b), 4) for b in [X1, X2]] for a in [X1, X2]]
      Sigma
```

```
[12]: [[2.0, 1.2], [1.2, 1.2]]
```

Correlation Coefficient

Given two random variables X and Y , their *correlation coefficient* is:

$$\text{Corr}(X, Y) = \frac{\text{Cov}(X, Y)}{\sigma_X \sigma_Y}$$

The correlation coefficient is always in the range $[-1, 1]$.

```
[13]: X, Y = [1, 2, 3, 4, 5], [2, 4, 5, 4, 5]
      n = len(X)
      mu_X, mu_Y = sum(X)/n, sum(Y)/n
      cov_xy = sum((X[i] - mu_X) * (Y[i] - mu_Y) for i in range(n)) / n
      sig_X = math.sqrt(sum((x - mu_X)**2 for x in X) / n)
      sig_Y = math.sqrt(sum((y - mu_Y)**2 for y in Y) / n)
      round(cov_xy / (sig_X * sig_Y), 6)
```

```
[13]: 0.774597
```

Entropy

The *entropy* of a random variable X is denoted $H(X)$. For discrete random variables:

$$H(X) = - \sum_x P(X = x) \log P(X = x)$$

where the sum is over all values x that X may take (and with the convention $0 \log 0 = 0$). For a continuous random variable:

$$H(X) = - \int_{-\infty}^{\infty} f(x) \log f(x) dx$$

where $f(x)$ is the PDF. The base of the logarithm depends on the application, with the most popular choices being 2, e , and 10.

Extensions of this notation include the *joint entropy* $H(X, Y)$ and the *conditional entropy* $H(X | Y)$.

```
[14]: # entropy of a fair die (base 2)
probs = [1/6] * 6
H_X = -sum(p * math.log2(p) for p in probs if p > 0)
print(f'H(X) = {H_X:.4f} bits')
print(f'log2(6) = {math.log2(6):.4f} bits (maximum entropy)')
```

H(X) = 2.5850 bits

log2(6) = 2.5850 bits (maximum entropy)

Distributions

Random variables are often best described by their *distribution*. Distributions, in turn, can be described in one of three principal ways, each with its own acronym.

The notation $X \sim \mathcal{D}$ indicates that the random variable X has distribution $\mathcal{D}$. For example, $X \sim N(0, 1)$ states that X is a standard normal random variable.

Probability Mass Function (PMF)

Discrete random variables can be specified by their *probability mass function*, or PMF, which specifies the probability associated with each possible value of the random variable:

$$f(x) = P(X = x)$$

```
[15]: # PMF of a fair die
pmf = {x: 1/6 for x in range(1, 7)}
for x, p in pmf.items():
    print(f'P(X = {x}) = {p:.4f}')
```

$P(X = 1) = 0.1667$
 $P(X = 2) = 0.1667$
 $P(X = 3) = 0.1667$
 $P(X = 4) = 0.1667$
 $P(X = 5) = 0.1667$
 $P(X = 6) = 0.1667$

Cumulative Distribution Function (CDF)

Real-valued, continuous random variables can be specified by their *cumulative distribution function*, or CDF:

$$F(x) = P(X \leq x)$$

Most authors denote cumulative distribution functions with uppercase letters. The inverse function of the CDF is the *quantile function*, which is often denoted Q .

```
[16]: # CDF of a fair die: F(x) = P(X <= x)
      for x in range(1, 7):
          F_x = x / 6
          print(f'F({x}) = P(X <= {x}) = {F_x:.4f}')
```

$F(1) = P(X \leq 1) = 0.1667$
 $F(2) = P(X \leq 2) = 0.3333$
 $F(3) = P(X \leq 3) = 0.5000$
 $F(4) = P(X \leq 4) = 0.6667$
 $F(5) = P(X \leq 5) = 0.8333$
 $F(6) = P(X \leq 6) = 1.0000$

Probability Density Function (PDF)

Real-valued, continuous random variables can also be specified by their *probability density function* $f(x)$. The relation to the cumulative distribution function $F(x)$ is $f(x) = F'(x)$, which gives:

$$P(X \in [a, b]) = \int_a^b f(x) dx$$

Most authors use lowercase letters that correspond to the uppercase letter for the random variable's CDF.

```
[17]: # standard normal PDF: phi(x) = (1/sqrt(2*pi)) * exp(-x^2 / 2)
      def phi(x):
          return math.exp(-x**2 / 2) / math.sqrt(2 * math.pi)
      for x in [-2, -1, 0, 1, 2]:
          print(f'phi({x:+d}) = {phi(x):.6f}')
```



```

phi(-2) = 0.053991
phi(-1) = 0.241971
phi(+0) = 0.398942
phi(+1) = 0.241971
phi(+2) = 0.053991

```

Identically Distributed and IID

A collection of random variables with the same distribution are called *identically distributed*. If a collection of random variables are (mutually) independent and identically distributed, they are said to be *iid* (or IID):

$$X_1, X_2, \dots, X_n \stackrel{\text{iid}}{\sim} \mathcal{D}$$

```

[18]: random.seed(42)
      # X_1, ..., X_5 iid ~ N(0, 1)
      X = [round(random.gauss(0, 1), 4) for _ in range(5)]
      print('IID samples from N(0, 1):', X)

```

IID samples from N(0, 1): [-0.1441, -0.1729, -0.1113, 0.702, -0.1276]

Binomial Distribution

Let n be a nonnegative integer and $0 \leq p \leq 1$. A discrete random variable X has the *binomial distribution* $\text{Bin}(n, p)$ or $B(n, p)$ if its probability mass function is:

$$P(X = k) = \binom{n}{k} p^k (1 - p)^{n-k}$$

where $k \in \{0, 1, \dots, n\}$.

```

[19]: n, p = 10, 0.3
      for k in range(n + 1):
          pmf = math.comb(n, k) * p**k * (1 - p)**(n - k)
          print(f'P(X = {k:2d}) = {pmf:.4f}')

```

```

P(X = 0) = 0.0282
P(X = 1) = 0.1211
P(X = 2) = 0.2335
P(X = 3) = 0.2668
P(X = 4) = 0.2001
P(X = 5) = 0.1029
P(X = 6) = 0.0368
P(X = 7) = 0.0090
P(X = 8) = 0.0014
P(X = 9) = 0.0001
P(X = 10) = 0.0000

```

Poisson Distribution

Let λ be a positive real number. A discrete random variable X has the *Poisson distribution* $\text{Pois}(\lambda)$ provided:

$$P(X = k) = \frac{\lambda^k e^{-\lambda}}{k!}$$

where k is a nonnegative integer. The use of λ for the parameter (and hence the mean) of the Poisson distribution is quite common.

```
[20]: lam = 3.0
      for k in range(10):
          pmf = lam**k * math.exp(-lam) / math.factorial(k)
          print(f'P(X = {k}) = {pmf:.4f}')
```

```
P(X = 0) = 0.0498
P(X = 1) = 0.1494
P(X = 2) = 0.2240
P(X = 3) = 0.2240
P(X = 4) = 0.1680
P(X = 5) = 0.1008
P(X = 6) = 0.0504
P(X = 7) = 0.0216
P(X = 8) = 0.0081
P(X = 9) = 0.0027
```

Exponential Distribution

For a positive real number α , the *exponential distribution* is denoted $\text{Exp}(\alpha)$. The CDF for a random variable with this distribution is:

$$F(t) = 1 - \exp(-\alpha t) \quad \text{where } t \geq 0$$

```
[21]: alpha = 2.0
      for t in [0.0, 0.25, 0.5, 1.0, 2.0, 5.0]:
          F_t = 1 - math.exp(-alpha * t)
          print(f'F({t:.2f}) = {F_t:.4f}')
```

```
F(0.00) = 0.0000
F(0.25) = 0.3935
F(0.50) = 0.6321
F(1.00) = 0.8647
F(2.00) = 0.9817
F(5.00) = 1.0000
```

Normal / Gaussian Distribution

A *standard normal* or *Gaussian* random variable X has a probability density function typically denoted $\phi(x)$:

$$\phi(x) = \frac{1}{\sqrt{2\pi}} \exp\left\{-\frac{x^2}{2}\right\}$$

The corresponding CDF is often denoted $\Phi(x)$. It is closely related to the error function $\text{erf}(x)$:

$$\Phi(x) = \frac{1}{2} \left[1 + \text{erf}\left(\frac{x}{\sqrt{2}}\right) \right]$$

For more information on the error function, see the [Functions notebook](#).

More generally, if μ is a real number and σ^2 is a positive real number, then $N(\mu, \sigma^2)$ is the distribution of a normal random variable X with mean μ and variance σ^2 . Its probability density function is:

$$\frac{1}{\sigma\sqrt{2\pi}} \exp\left\{-\frac{(x-\mu)^2}{2\sigma^2}\right\}$$

A standard normal random variable has $X \sim N(0, 1)$.

The multivariable normal random variable $N_k(\mu, \Sigma)$ is a random k -vector with mean μ and covariance matrix Σ .

```
[22]: # standard normal PDF phi(x) and CDF Phi(x)
def phi(x):
    return math.exp(-x**2 / 2) / math.sqrt(2 * math.pi)
def Phi(x):
    return (1 + math.erf(x / math.sqrt(2))) / 2
for x in [-2, -1, 0, 1, 2]:
    print(f'x = {x:+d}: phi(x) = {phi(x):.6f}, Phi(x) = {Phi(x):.6f}')
```

```
x = -2: phi(x) = 0.053991, Phi(x) = 0.022750
x = -1: phi(x) = 0.241971, Phi(x) = 0.158655
x = +0: phi(x) = 0.398942, Phi(x) = 0.500000
x = +1: phi(x) = 0.241971, Phi(x) = 0.841345
x = +2: phi(x) = 0.053991, Phi(x) = 0.977250
```

The general normal distribution $N(\mu, \sigma^2)$ with arbitrary mean and variance:

```
[23]: mu, sigma = 5.0, 2.0
def normal_pdf(x, mu, sigma):
    return math.exp(-(x - mu)**2 / (2 * sigma**2)) / (sigma * math.sqrt(2 *
↳math.pi))
for x in [1, 3, 5, 7, 9]:
    print(f'f({x}) = {normal_pdf(x, mu, sigma):.6f}')
```

```
f(1) = 0.026995
f(3) = 0.120985
f(5) = 0.199471
f(7) = 0.120985
f(9) = 0.026995
```

Chi-Squared Distribution

The *chi-squared* distribution is denoted χ^2 ; more specifically, the chi-squared distribution with k degrees of freedom is denoted χ_k^2 . If $X_1, X_2, \dots, X_k$ are IID standard normal random variables, then the sum of their squares $X = X_1^2 + \dots + X_k^2$ is chi-squared distributed χ_k^2 . Its probability density function is:

$$f(x) = \frac{1}{2^{k/2} \Gamma(k/2)} x^{k/2-1} e^{-x/2}$$

where Γ is the gamma function, satisfying $\Gamma(n) = (n-1)!$ for positive integers.

For more information on the gamma function, see the [Functions](#) notebook.

```
[24]: k = 5
def chi2_pdf(x, k):
    return x**(k/2 - 1) * math.exp(-x/2) / (2**(k/2) * math.gamma(k/2))
for x in [1, 3, 5, 7, 10]:
    print(f'f({x}) = {chi2_pdf(x, k):.6f}')
```

```
f(1) = 0.080657
f(3) = 0.154180
f(5) = 0.122042
f(7) = 0.074371
f(10) = 0.028335
```

Student's t -Distribution

This distribution is denoted t_ν , where ν is a positive integer. It is given by $Z / \sqrt{V/\nu}$ where Z and V are independent random variables, Z is a standard normal, and V is χ_ν^2 . Its probability density function is:

$$f(x) = \frac{\Gamma(\frac{\nu+1}{2})}{\sqrt{\nu\pi} \Gamma(\frac{\nu}{2})} \left(1 + \frac{x^2}{\nu}\right)^{-(\nu+1)/2}$$

```
[25]: nu = 5
def t_pdf(x, nu):
    c = math.gamma((nu + 1) / 2) / (math.sqrt(nu * math.pi) * math.gamma(nu / 2))
    return c * (1 + x**2 / nu) ** (-(nu + 1) / 2)
for x in [-2, -1, 0, 1, 2]:
```

```
print(f'f({x:+d}) = {t_pdf(x, nu):.6f}')
```

```
f(-2) = 0.065090
f(-1) = 0.219680
f(+0) = 0.379607
f(+1) = 0.219680
f(+2) = 0.065090
```

F-Distribution

Also known as the Fisher–Snedecor distribution, this is written $F(\nu_1, \nu_2)$ where the ν_i are positive integers. It is defined as:

$$F = \frac{X_1/\nu_1}{X_2/\nu_2}$$

where X_1, X_2 are independent χ^2 -random variables with ν_1, ν_2 degrees of freedom, respectively.

```
[26]: random.seed(42)
v1, v2 = 5, 10
# simulate F-distributed values from chi-squared ratio
for _ in range(5):
    X1 = sum(random.gauss(0, 1)**2 for _ in range(v1))
    X2 = sum(random.gauss(0, 1)**2 for _ in range(v2))
    print(f'F = {(X1 / v1) / (X2 / v2):.4f}')
```

```
F = 0.2344
F = 1.0063
F = 3.8158
F = 0.2941
F = 1.0991
```

Convergence of Random Variables

Let X and $X_1, X_2, \dots$ be random variables. There are various notions of the sequence X_n converging to X , each with its own notation. Some authors place the notation as a subscript to the arrow rather than above, e.g., $X_n \rightarrow_p X$.

Convergence in Distribution

Convergence in distribution (also called *convergence in law*) is denoted in various ways:

$$X_n \xrightarrow{d} X \quad X_n \xrightarrow{D} X \quad X_n \xrightarrow{L} X \quad \mathcal{L}(X_n) \rightarrow \mathcal{L}(X)$$

The Central Limit Theorem is a classic result: if $X_1, X_2, \dots$ are IID with mean μ and variance σ^2 , then:

$$\frac{\sqrt{n}(\bar{X}_n - \mu)}{\sigma} \xrightarrow{d} N(0, 1)$$

```
[27]: random.seed(42)
# CLT: standardized mean of Uniform(0,1) approaches N(0,1)
for n in [10, 100, 1000]:
    zs = [(sum(random.random() for _ in range(n))/n - 0.5) * (12*n)**0.5
           for _ in range(500)]
    within = sum(1 for z in zs if abs(z) < 1.96) / 500
    print(f'n = {n:4d}: P(|Z| < 1.96) = {within:.3f} (expect ~0.950)')
```

```
n = 10: P(|Z| < 1.96) = 0.960 (expect ~0.950)
n = 100: P(|Z| < 1.96) = 0.934 (expect ~0.950)
n = 1000: P(|Z| < 1.96) = 0.960 (expect ~0.950)
```

Convergence in Probability

Convergence in probability is denoted:

$$X_n \xrightarrow{P} X \quad \text{or} \quad X_n \xrightarrow{P} X$$

This means that for every $\varepsilon > 0$, $P(|X_n - X| > \varepsilon) \rightarrow 0$ as $n \rightarrow \infty$.

```
[28]: random.seed(42)
epsilon = 0.1
for n in [10, 50, 100, 500]:
    viols = sum(1 for _ in range(1000)
                if abs(sum(random.random() for _ in range(n))/n - 0.5) >
                    epsilon)
    print(f'n = {n:3d}: P(|X_bar - mu| > {epsilon}) = {viols/1000:.3f}')
```

```
n = 10: P(|X_bar - mu| > 0.1) = 0.287
n = 50: P(|X_bar - mu| > 0.1) = 0.010
n = 100: P(|X_bar - mu| > 0.1) = 0.001
n = 500: P(|X_bar - mu| > 0.1) = 0.000
```

Almost Sure Convergence

Almost sure convergence is denoted:

$$X_n \xrightarrow{a.s.} X$$

Almost sure convergence implies convergence in probability. The strong law of large numbers states that $\bar{X}_n \xrightarrow{a.s.} \mu$.

```
[29]: random.seed(42)
# strong law: running average converges to mu = 0.5
values = [random.random() for _ in range(10000)]
for n in [1, 10, 100, 1000, 10000]:
    x_bar = sum(values[:n]) / n
    print(f'n = {n:5d}: X_bar = {x_bar:.6f} (mu = 0.500000)')
```

```
n =      1: X_bar = 0.639427 (mu = 0.500000)
n =     10: X_bar = 0.400669 (mu = 0.500000)
n =    100: X_bar = 0.479569 (mu = 0.500000)
n =   1000: X_bar = 0.512562 (mu = 0.500000)
n =  10000: X_bar = 0.500251 (mu = 0.500000)
```

L^2 Convergence

L^2 convergence is denoted:

$$X_n \xrightarrow{L^2} X$$

This means $E[(X_n - X)^2] \rightarrow 0$ as $n \rightarrow \infty$. L^2 convergence implies convergence in probability.

```
[30]: random.seed(42)
# E[(X_bar - mu)^2] -> 0 as n -> infinity
for n in [10, 50, 100, 500]:
    mse = sum((sum(random.random() for _ in range(n))/n - 0.5)**2
               for _ in range(1000)) / 1000
    print(f'n = {n:3d}: E[(X_bar - mu)^2] = {mse:.6f}')
```

```
n =  10: E[(X_bar - mu)^2] = 0.008277
n =  50: E[(X_bar - mu)^2] = 0.001689
n = 100: E[(X_bar - mu)^2] = 0.000858
n = 500: E[(X_bar - mu)^2] = 0.000161
```

§9.2 Statistics

Given numerical data $X_1, X_2, \dots, X_n$, the following notations are commonly used in statistics.

Average

The (sample) *average* is denoted $\bar{X}$:

$$\bar{X} = \frac{1}{n} \sum_{i=1}^n X_i$$

```
[31]: X = [4, 7, 13, 2, 9]
      X_bar = sum(X) / len(X)
      X_bar
```

```
[31]: 7.0
```

Median

The (sample) *median* is denoted $\tilde{X}$ or $\hat{X}$. It is the middle value when the data are sorted. For an even number of observations, the median is the average of the two middle values.

```
[32]: X = [4, 7, 13, 2, 9]
      X_sorted = sorted(X)
      n = len(X_sorted)
      median = X_sorted[n // 2] if n % 2 == 1 else (X_sorted[n//2 - 1] + X_sorted[n//
↪2]) / 2
      print('sorted:', X_sorted, ', median:', median)
```

```
sorted: [2, 4, 7, 9, 13] , median: 7
```

Sample Standard Deviation

The (sample) *standard deviation* is denoted s :

$$s = \sqrt{\frac{\sum_{i=1}^n (X_i - \bar{X})^2}{n - 1}}$$

The denominator $n - 1$ (rather than n) gives an unbiased estimate of the population variance.

```
[33]: X = [4, 7, 13, 2, 9]
      X_bar = sum(X) / len(X)
      s = math.sqrt(sum((x - X_bar)**2 for x in X) / (len(X) - 1))
      s
```

```
[33]: 4.301162633521313
```

z -Score

A common way to standardize data is to express it as the number of standard deviations above (or below) the mean. For this one uses the *z-score*:

$$Z_i = \frac{X_i - \mu}{\sigma}$$

where μ is the population mean and σ is the population standard deviation.


```
[34]: X = [4, 7, 13, 2, 9]
mu = sum(X) / len(X)
sigma = (sum((x - mu)**2 for x in X) / len(X)) ** 0.5
Z = [round((x - mu) / sigma, 4) for x in X]
print('z-scores:', Z)
```

z-scores: [-0.7798, 0.0, 1.5596, -1.2997, 0.5199]

Order Statistics

The notation $X_{(1)}, X_{(2)}, \dots, X_{(n)}$ represents the same values as $X_1, \dots, X_n$ but in nondecreasing order. That is, $X_{(1)}$ is the smallest X_i , $X_{(n)}$ is the largest, and in general:

$$X_{(1)} \leq X_{(2)} \leq \dots \leq X_{(n)}$$

```
[35]: X = [4, 7, 13, 2, 9]
X_ordered = sorted(X)
for i, x in enumerate(X_ordered, 1):
    print(f'X_{(i)} = {x}')
```

X_(1) = 2
X_(2) = 4
X_(3) = 7
X_(4) = 9
X_(5) = 13

Hypothesis Testing

In hypothesis testing, the *null hypothesis* is typically denoted H_0 and the *alternative hypothesis* by H_A or H_1 .

Incorrectly rejecting H_0 when it is true is known as a *type I error*, and the probability of committing such a mistake is typically denoted α .

Accepting H_0 when it is false is known as a *type II error*, and the probability of this mistake is typically denoted β .

```
[36]: # H_0: p = 0.5 (fair coin), H_1: p != 0.5
alpha = 0.05
n, heads = 100, 62
z = (heads/n - 0.5) / (0.5 / n**0.5)
print(f'z = {z:.3f}, reject H_0 at alpha = {alpha}: {abs(z) > 1.96}')
```

z = 2.400, reject H_0 at alpha = 0.05: True

Error Terms

In statistical models, error (or noise) terms are often denoted with the letter e or ε . These can be subscripted if there are multiple error terms:

$$Y_i = \beta_0 + \beta_1 X_i + \varepsilon_i$$

```
[37]: random.seed(42)
# Y = 2X + 3 + epsilon, where epsilon ~ N(0, 0.5)
X = [1, 2, 3, 4, 5]
epsilon = [round(random.gauss(0, 0.5), 3) for _ in X]
Y = [round(2*x + 3 + e, 3) for x, e in zip(X, epsilon)]
print('epsilon:', epsilon)
print('Y:      ', Y)
```

```
epsilon: [-0.072, -0.086, -0.056, 0.351, -0.064]
Y:      [4.928, 6.914, 8.944, 11.351, 12.936]
```

Estimator

When one is trying to determine some numerical quantity for a population and this is done by computing an estimate from a random sample, it is conventional to place a hat on the estimator. That is, if θ is the parameter whose value is sought, then $\hat{\theta}$ is the estimator of θ .

```
[38]: random.seed(42)
# true parameter theta = 5.0, estimate from sample mean
theta = 5.0
sample = [random.gauss(theta, 1) for _ in range(50)]
theta_hat = sum(sample) / len(sample)
print(f'theta = {theta}, theta_hat = {theta_hat:.4f}')
```

```
theta = 5.0, theta_hat = 4.9420
```